DARSHAN PRAJAPATI

MCA-1

ROLL NO: 24

RDBMS PRACTICAL ASSIGNMENT-1

```
use practical_1;


# create tables:

create table Deposit(

ACTNO VARCHAR(5),

CNAME VARCHAR(18),

BNAME VARCHAR(18),

AMOUNT DECIMAL(8,2),

ADATE DATE

);


create table BORROW(

LOANNO VARCHAR(5),

CNAME VARCHAR(18),

BNAME VARCHAR(18),

AMOUNT DECIMAL(8,2)

);


CREATE TABLE CUSTOMER(
```

```
CNAME VARCHAR(18),

CITY VARCHAR(18));


CREATE TABLE BRANCH(

BNAME VARCHAR(18),

CITY VARCHAR(18));



# Insert values into talbes

INSERT INTO deposit VALUES

(100, 'ANIL', 'VRCE', 1000, '1995-03-01'),

(101, 'SUNIL', 'AJNI', 5000, '1998-01-04'),

(102, 'MEHUL', 'KAROLBAGH', 3500, '1995-11-17'),

(104, 'MADHURI', 'CHANDNI', 1200, '1995-12-17'),

(105, 'PRAMOD', 'MGROAD', 3000, '1996-03-27'),

(106, 'SANDIP', 'ANDHERI', 2000, '1996-03-31'),

(107, 'SHIVANI', 'VIRAR', 1000, '1995-09-05'),

(108, 'KRANTI', 'NEHRUPLACE', 5000, '1995-07-02'),

(109, 'NAREN', 'POWAI', 7000, '1995-10-10');


SELECT * FROM DEPOSIT;


INSERT INTO customer VALUES
```

```
('ANIL', 'CALCUTTA'),
('SUNIL', 'DELHI'),
('MEHUL', 'BARODA'),
('MANDAR', 'PATNA'),
('MADHURI', 'NAGPUR'),
('PRAMOD', 'NAGPUR'),
('SANDIP', 'SURAT'),
('SHIVANI', 'BOMBAY'),
('KRANTI', 'BOMBAY'),
('NAREN', 'BOMBAY');

select * from customer;

INSERT INTO branch VALUES
('VRCE', 'NAGPUR'),
('AJNI', 'NAGPUR'),
('KAROLBAGH', 'DELHI'),
('CHANDNI', 'DELHI'),
('DHARAMPETH', 'NAGPUR'),
('MGROAD', 'BANGALORE'),
('ANDHERI', 'BOMBAY'),
('VIRAR', 'BOMBAY'),
('NEHRUPLACE', 'DELHI'),
```

```
('POWAI', 'BOMBAY');

select * from branch;

INSERT INTO borrow VALUES
(201, 'ANIL', 'VRCE', 1000),
(206, 'MEHUL', 'AJNI', 5000),
(311, 'SUNIL', 'DHARAMPETH', 3000),
(321, 'MADHURI', 'ANDHERI', 2000),
(375, 'PRAMOD', 'VIRAR', 8000),
(481, 'KRANTI', 'NEHRUPLACE', 3000);

select * from borrow;
```

Questions:

/*

1. Display all the deposit details of customer Anil.

2. List all customers who are living in Bombay.

3. Show depositors who have deposited more than 3000.

4. Display borrowers who have taken a loan of less than 5000.

5. Find all depositors from Nagpur branch.

6. Show the total deposit amount in the bank.

7. Find the maximum deposit amount and the customer who made it.

8. Display the average loan amount of all borrowers.

9. Show the number of customers living in each city.

10. Find the total deposit amount branch–wise.

11. List depositors who deposited between 2000 and 6000.

12. Display borrowers who belong to the branch Andheri.

13. Find the minimum loan amount and the customer who took it.

14. Show the total number of depositors.

15. List all customers whose account date is after 01-JAN-1996.

*/

1. Display all the deposit details of customer Anil.

```
select * from deposit where cname='anil';
```

2. List all customers who are living in Bombay.

```
select * from customer where city='bombay';
```

3. Show depositors who have deposited more than 3000.

```
select cname,amount from deposit where amount>3000;
```

4. Display borrowers who have taken a loan of less than 5000.

```
select cname, amount from borrow where amount<5000;
```

5. Find all depositors from Nagpur branch.

select cname from deposit where bname in (select bname from branch where city='nagpur')
;

6. Show the total deposit amount in the bank.

select sum(amount) as Total_deposit from deposit ;

7. Find the maximum deposit amount and the customer who made it.

select amount , cname from deposit where amount=(select max(amount) from deposit);

8. Display the average loan amount of all borrowers.

select avg(amount) as Avg_loan_amount from borrow;

9. Show the number of customers living in each city.

select city, count(cname) as number_of_customers from customer group by city;

10. Find the total deposit amount branch–wise.

select bname,sum(amount) from deposit group by bname order by bname;

11. List depositors who deposited between 2000 and 6000.

select cname, amount from deposit where amount between 2000 and 6000;

12. Display borrowers who belong to the branch Andheri.

select cname from borrow where bname='andheri';

13. Find the minimum loan amount and the customer who took it.

```
select amount, cname from borrow where amount=(select min(amount) from borrow ) ;
```

14. Show the total number of depositors.

```
select count(distinct cname) as Number_of_depositors from deposit;
```

15. List all customers whose account date is after 01-JAN-1996.

```
select cname , adate  from deposit where adate > '1996-01-01' ;
```

RDBMS PRACTICAL ASSIGNMENT-2

```
use practical_2;

-- Customer Table

CREATE TABLE Customer (
CustID INT PRIMARY KEY,
Name VARCHAR(30),
City VARCHAR(20)
);

-- Account Table
```

```
CREATE TABLE Account (

AccNo INT PRIMARY KEY,

CustID INT,

Balance DECIMAL(10,2),

BranchName VARCHAR(20),

FOREIGN KEY (CustID) REFERENCES Customer(CustID)

);


-- Loan Table


CREATE TABLE Loan (

LoanNo INT PRIMARY KEY,

CustID INT,

Amount DECIMAL(10,2),

BranchName VARCHAR(20),

FOREIGN KEY (CustID) REFERENCES Customer(CustID)

);


-- Branch Table


CREATE TABLE Branch (

BranchName VARCHAR(20) PRIMARY KEY,
```

```
City VARCHAR(20)
);

-- Insert Sample Data
-- Customers

INSERT INTO Customer VALUES
(1, 'Anil', 'Nagpur'),
(2, 'Sunita', 'Mumbai'),
(3, 'Mehul', 'Delhi'),
(4, 'Priya', 'Nagpur'),
(5, 'Naren', 'Pune');

-- Branches

INSERT INTO Branch VALUES
('VRCE', 'Nagpur'),
('AJNI', 'Nagpur'),
('KAROLBAGH', 'Delhi'),
('ANDHERI', 'Mumbai'),
('MGROAD', 'Pune');

-- Accounts
```

```
INSERT INTO Account VALUES

(1001, 1, 5000, 'VRCE'),

(1002, 2, 8000, 'AJNI'),

(1003, 3, 12000, 'KAROLBAGH'),

(1004, 4, 3000, 'ANDHERI'),

(1005, 5, 15000, 'MGROAD');


-- Loans


INSERT INTO Loan VALUES

(2001, 1, 4000, 'VRCE'),

(2002, 2, 6000, 'AJNI'),

(2003, 3, 10000, 'KAROLBAGH'),

(2004, 4, 2000, 'ANDHERI'),

(2005, 5, 12000, 'MGROAD');


-- Assignment Questions (Simplified Subqueries & Nested Queries)

/*

1.Find customers who have balance more than 8000.

2.List customers who have taken a loan amount greater than 5000.

3.Find customers who have an account in the Nagpur branches.

4.Find the customer(s) who have the maximum loan amount.
```

5.Find the branch where Sunita has her account.

6.Find customers who have a loan in the Delhi branch.

7.Find customers whose balance is equal to 5000.

8.Find the loan amount of Priya.

9.Find customers who belong to the same city as Naren.

10.Find the branch name where Anil has his loan.

11.Find all customers whose balance is less than Sunita's balance.

12.Find the customer who has the minimum loan amount.

13.Find customers whose loan amount is equal to their account balance.

14.Find customers who do not have a loan.

15.Find customers who do not have an account.

*/

```
-- 1.Find customers who have balance more than 8000.

select * from customer where custid in (select custid from account where balance>8000);


-- 2.List customers who have taken a loan amount greater than 5000.

select * from customer where custid in (select custid from loan where amount>5000);


-- 3.Find customers who have an account in the Nagpur branches.

select name from customer where custid in

(select custid from account where branchname in

(select branchname from branch where city='Nagpur')
```

```
);

-- 4.Find the customer(s) who have the maximum loan amount.
SELECT Name
FROM Customer
WHERE CustID IN (
    SELECT CustID
    FROM Loan
    WHERE Amount = (SELECT MAX(Amount) FROM Loan)
);

-- 5.Find the branch where Sunita has her account.
-- custid where customer.name= sunita
-- branchname where account.custid =customer.custid

SELECT BranchName
FROM Account
WHERE CustID = (
    SELECT CustID FROM Customer WHERE Name = 'Sunita'
);

-- 6.Find customers who have a loan in the Delhi branch.
SELECT Name
```

```
FROM Customer
WHERE CustID IN (
    SELECT CustID
    FROM Loan
    WHERE BranchName IN (
        SELECT BranchName
        FROM Branch
        WHERE City = 'Delhi'
    )
);

-- 7.Find customers whose balance is equal to 5000.
SELECT Name
FROM Customer
WHERE CustID IN (
    SELECT CustID
    FROM Account
    WHERE Balance = 5000
);

-- 8.Find the loan amount of Priya.
SELECT Amount
FROM Loan
```

```
WHERE CustID = (
    SELECT CustID FROM Customer WHERE Name = 'Priya'
);
```

-- 9.Find customers who belong to the same city as Naren.

```
SELECT Name
FROM Customer
WHERE City = (
    SELECT City FROM Customer WHERE Name = 'Naren'
)
AND Name <> 'Naren';
```

-- 10.Find the branch name where Anil has his loan.

```
SELECT BranchName
FROM Loan
WHERE CustID = (
    SELECT CustID FROM Customer WHERE Name = 'Anil'
);
```

-- 11.Find all customers whose balance is less than Sunita’s balance.

```
SELECT Name
FROM Customer
WHERE CustID IN (
```

```
    SELECT CustID
    FROM Account
    WHERE Balance < (
        SELECT Balance
        FROM Account
        WHERE CustID = (SELECT CustID FROM Customer WHERE Name = 'Sunita')
    )
);

-- 12.Find the customer who has the minimum loan amount.
SELECT Name
FROM Customer
WHERE CustID IN (
    SELECT CustID
    FROM Loan
    WHERE Amount = (SELECT MIN(Amount) FROM Loan)
);

-- 13.Find customers whose loan amount is equal to their account balance.
SELECT Name
FROM Customer
WHERE CustID IN (
    SELECT A.CustID
```

```
    FROM Account A
    JOIN Loan L ON A.CustID = L.CustID
    WHERE A.Balance = L.Amount
);

-- 14.Find customers who do not have a loan.
SELECT Name
FROM Customer
WHERE CustID NOT IN (SELECT CustID FROM Loan);

-- 15.Find customers who do not have an account.
SELECT Name
FROM Customer
WHERE CustID NOT IN (SELECT CustID FROM Account);
```

RDBMS PRACTICAL ASSIGNMENT-3

```
use a3;

/* create tables */
create table customer(
CustID int primary key,
```

```
Name varchar(30),
City Varchar(20)
);

create table Account(
AccNo Int Primary key,
CustID int,
Balance decimal(10,2),
BranchName varchar(20),
Foreign key(CustID) references Customer(CustID));

create table Loan(
LoanNo int primary key,
CustId int,
Amount Decimal(10,2),
BranchName varchar(20),
foreign key (CustId) references Customer(CustId));

create table Branch(
BranchName varchar(20),
City Varchar(20));

/* insert values */
INSERT INTO Customer VALUES
(1, 'Anil', 'Nagpur'),
```

```
(2, 'Sunita', 'Mumbai'),
(3, 'Mehul', 'Delhi'),
(4, 'Priya', 'Nagpur'),
(5, 'Naren', 'Pune');

INSERT INTO Branch VALUES
('VRCE', 'Nagpur'),
('AJNI', 'Nagpur'),
('KAROLBAGH', 'Delhi'),

('ANDHERI', 'Mumbai'),
('MGROAD', 'Pune');

INSERT INTO Account VALUES
(1001, 1, 5000, 'VRCE'),
(1002, 2, 8000, 'AJNI'),
(1003, 3, 12000, 'KAROLBAGH'),
(1004, 4, 3000, 'ANDHERI'),
(1005, 5, 15000, 'MGROAD');

INSERT INTO Loan VALUES
(2001, 1, 4000, 'VRCE'),
(2002, 2, 6000, 'AJNI'),
(2003, 3, 10000, 'KAROLBAGH'),
(2004, 4, 2000, 'ANDHERI'),
```

```
(2005, 5, 12000, 'MGROAD');


-- Simple Inner Query Practice

-- 1. Display all balances from the Account table.

SELECT Balance FROM Account;


-- 2. Display all loan amounts from the Loan table.

SELECT Amount FROM Loan;


-- 3. Find the maximum loan amount.

SELECT MAX(Amount) AS Maximum_Loan FROM Loan;


-- 4. Find the minimum account balance.

SELECT MIN(Balance) AS Minimum_Balance FROM Account;


-- 5. Find the average account balance.

SELECT AVG(Balance) AS Avg_Balance FROM Account;


-- 6. Display all branch names located in ‘Nagpur’.

SELECT BranchName FROM Branch WHERE City = 'Nagpur';


-- Subqueries using ANY

-- 7. Display customers whose account balance is greater than ANY loan amount.

SELECT Name

FROM Customer
```

```
WHERE CustID IN (
  SELECT CustID
  FROM Account
  WHERE Balance > ANY (SELECT Amount FROM Loan)
);
```

-- 8. Display customers whose account balance is less than ANY loan amount.

```
SELECT Name
FROM Customer
WHERE CustID IN (
  SELECT CustID
  FROM Account
  WHERE Balance < ANY (SELECT Amount FROM Loan)
);
```

-- 9. Find customers whose loan amount is greater than ANY account balance.

```
SELECT Name
FROM Customer
WHERE CustID IN (
  SELECT CustID
  FROM Loan
  WHERE Amount > ANY (SELECT Balance FROM Account)
);
```

-- 10. Display customers whose loan amount is greater than ANY loan amount taken in ‘AJNI’ branch.

```
SELECT Name
FROM Customer
WHERE CustID IN (
  SELECT CustID
  FROM Loan
  WHERE Amount > ANY (
    SELECT Amount
    FROM Loan
    WHERE BranchName = 'AJNI'
  )
);
```

-- 11. Display customers whose account balance is greater than ANY account balance in ‘Nagpur’ branches.

```
SELECT Name
FROM Customer
WHERE CustID IN (
  SELECT CustID
  FROM Account
  WHERE Balance > ANY (
    SELECT Balance
    FROM Account
    WHERE BranchName IN (
      SELECT BranchName FROM Branch WHERE City = 'Nagpur'
    )
```

```
  )
);

-- 12. Display customers whose loan amount is greater than ANY balance of customers in the ‘Pune’ branches.
SELECT Name
FROM Customer
WHERE CustID IN (
  SELECT CustID
  FROM Loan
  WHERE Amount > ANY (
    SELECT Balance
    FROM Account
    WHERE BranchName IN (
      SELECT BranchName FROM Branch WHERE City = 'Pune'
    )
  )
);

-- Subqueries using ALL
-- 13. Display customers whose account balance is greater than ALL loan amounts.
SELECT Name
FROM Customer
WHERE CustID IN (
  SELECT CustID
```

```
  FROM Account
  WHERE Balance > ALL (SELECT Amount FROM Loan)
);

-- 14. Display customers whose loan amount is greater than ALL account balances.
SELECT Name
FROM Customer
WHERE CustID IN (
  SELECT CustID
  FROM Loan
  WHERE Amount > ALL (SELECT Balance FROM Account)
);

-- 15. Find customers whose account balance is less than ALL loan amounts.
SELECT Name
FROM Customer
WHERE CustID IN (
  SELECT CustID
  FROM Account
  WHERE Balance < ALL (SELECT Amount FROM Loan)
);

-- 16. Display customers whose loan amount is greater than ALL loan amounts taken in 'KAROLBAGH' branch.
SELECT Name
```

```
FROM Customer
WHERE CustID IN (
  SELECT CustID
  FROM Loan
  WHERE Amount > ALL (
    SELECT Amount
    FROM Loan
    WHERE BranchName = 'KAROLBAGH'
  )
);
```

-- 17. Find customers whose account balance is greater than ALL account balances in ‘ANDHERI’ branch.

```
SELECT Name
FROM Customer
WHERE CustID IN (
  SELECT CustID
  FROM Account
  WHERE Balance > ALL (
    SELECT Balance
    FROM Account
    WHERE BranchName = 'ANDHERI'
  )
);
```

```
-- 18. Display customers whose loan amount is greater than ALL loan amounts of ‘Nagpur’ branches.
SELECT Name
FROM Customer
WHERE CustID IN (
  SELECT CustID
  FROM Loan
  WHERE Amount > ALL (
    SELECT Amount
    FROM Loan
    WHERE BranchName IN (
      SELECT BranchName FROM Branch WHERE City = 'Nagpur'
    )
  )
);
```

RDBMS PRACTICAL ASSIGNMENT-4

```
create schema a4;
use a4;

-- Concept 1 — Cursors (8 Questions)

-- Q1. Create a table `students(id, name, marks)` and write a cursor to display all student
names one by one.
create table students(
```

```
id int primary key auto_increment,

name varchar(50),

marks int );


INSERT INTO students(name, marks) VALUES

('Amit', 85),

('Bhavin', 72),

('Chirag', 60),

('Disha', 45),

('Ekta', 90);

select * from students;


-- display student names


DELIMITER $$


CREATE PROCEDURE show_student_names()

BEGIN

  DECLARE done INT DEFAULT 0;

  DECLARE v_name VARCHAR(50);


  DECLARE cur CURSOR FOR

    SELECT name FROM students;
```

```
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

  OPEN cur;

  read_loop: LOOP
    FETCH cur INTO v_name;
    IF done = 1 THEN
      LEAVE read_loop;
    END IF;

    -- print one name
    SELECT v_name AS student_name;
  END LOOP;

  CLOSE cur;
END $$

DELIMITER ;

CALL show_student_names();

-- Q2. Write a cursor that fetches both the name and marks of students and prints them.
```

```
DELIMITER $$

CREATE PROCEDURE show_student_name_marks()
BEGIN
  DECLARE done INT DEFAULT 0;
  DECLARE v_name  VARCHAR(50);
  DECLARE v_marks INT;

  DECLARE cur CURSOR FOR
    SELECT name, marks FROM students;

  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

  OPEN cur;

  read_loop: LOOP
    FETCH cur INTO v_name, v_marks;
    IF done = 1 THEN
      LEAVE read_loop;
    END IF;

    SELECT v_name AS name, v_marks AS marks;
```

```
  END LOOP;

  CLOSE cur;
END $$

DELIMITER ;

CALL show_student_name_marks();
```

-- Q3. Write a cursor to classify each student into grade A, B, or C based on marks.

```
DELIMITER $$

CREATE PROCEDURE show_student_grades()
BEGIN
  DECLARE done INT DEFAULT 0;
  DECLARE v_name  VARCHAR(50);
  DECLARE v_marks INT;
  DECLARE v_grade CHAR(1);

  DECLARE cur CURSOR FOR
```

```
  SELECT name, marks FROM students;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

OPEN cur;

read_loop: LOOP
  FETCH cur INTO v_name, v_marks;
  IF done = 1 THEN
    LEAVE read_loop;
  END IF;

  IF v_marks >= 80 THEN
    SET v_grade = 'A';
  ELSEIF v_marks >= 60 THEN
    SET v_grade = 'B';
  ELSE
    SET v_grade = 'C';
  END IF;

  SELECT v_name AS name, v_marks AS marks, v_grade AS grade;
END LOOP;
```

```
  CLOSE cur;
END $$

DELIMITER ;

CALL show_student_grades();
```

-- Q4. Write a cursor that increases each student's marks by 5 and updates the table.

```
DELIMITER $$

CREATE PROCEDURE increase_marks_by_5()
BEGIN
  DECLARE done INT DEFAULT 0;
  DECLARE v_id    INT;
  DECLARE v_marks INT;

  DECLARE cur CURSOR FOR
    SELECT id, marks FROM students;

  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

  OPEN cur;
```

```
  read_loop: LOOP
    FETCH cur INTO v_id, v_marks;
    IF done = 1 THEN
      LEAVE read_loop;
    END IF;

    UPDATE students
    SET marks = v_marks + 5
    WHERE id = v_id;
  END LOOP;

  CLOSE cur;
END $$

DELIMITER ;

CALL increase_marks_by_5();

SELECT * FROM students;
```

-- Q5. Write a cursor to copy all students with marks ≥ 70 into `passed_students`.

```
DROP TABLE IF EXISTS passed_students;

CREATE TABLE passed_students (
  id    INT PRIMARY KEY,
  name  VARCHAR(50),
  marks INT
);

DELIMITER $$

DROP PROCEDURE IF EXISTS copy_passed_students $$

CREATE PROCEDURE copy_passed_students()
BEGIN
  DECLARE done   INT DEFAULT 0;
  DECLARE v_id    INT;
  DECLARE v_name  VARCHAR(50);
  DECLARE v_marks INT;

  DECLARE cur CURSOR FOR
    SELECT id, name, marks FROM students WHERE marks >= 70;
```

```
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

  OPEN cur;

  read_loop: LOOP
    FETCH cur INTO v_id, v_name, v_marks;
    IF done = 1 THEN
      LEAVE read_loop;
    END IF;

    INSERT IGNORE INTO passed_students(id, name, marks)
    VALUES (v_id, v_name, v_marks);
  END LOOP;

  CLOSE cur;
END $$

DELIMITER ;

CALL copy_passed_students();

SELECT * FROM passed_students;
```

```
-- Q6. Write a cursor to count the total number of students and display the count.

DELIMITER $$


CREATE PROCEDURE count_students()

BEGIN

  DECLARE done INT DEFAULT 0;

  DECLARE v_id INT;

  DECLARE v_count INT DEFAULT 0;


  DECLARE cur CURSOR FOR

    SELECT id FROM students;


  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;


  OPEN cur;


  read_loop: LOOP

    FETCH cur INTO v_id;

    IF done = 1 THEN

      LEAVE read_loop;

    END IF;


    SET v_count = v_count + 1;
```

```
  END LOOP;

  CLOSE cur;

  SELECT v_count AS total_students;
END $$

DELIMITER ;

CALL count_students();

-- Q7. Write a parameterized cursor simulation to fetch students in a marks range.
DELIMITER $$

CREATE PROCEDURE students_in_marks_range(IN p_min INT, IN p_max INT)
BEGIN
  DECLARE done INT DEFAULT 0;
  DECLARE v_name  VARCHAR(50);
  DECLARE v_marks INT;

  DECLARE cur CURSOR FOR
    SELECT name, marks
    FROM students
```

```
    WHERE marks BETWEEN p_min AND p_max;

  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

  OPEN cur;

  read_loop: LOOP
    FETCH cur INTO v_name, v_marks;
    IF done = 1 THEN
      LEAVE read_loop;
    END IF;

    SELECT v_name AS name, v_marks AS marks;
  END LOOP;

  CLOSE cur;
END $$

DELIMITER ;

CALL students_in_marks_range(60, 90);
```

```
-- Q8. Write a cursor that prints running maximum marks while scanning rows.


DELIMITER $$


CREATE PROCEDURE running_max_marks()
BEGIN
  DECLARE done INT DEFAULT 0;
  DECLARE v_name       VARCHAR(50);
  DECLARE v_marks      INT;
  DECLARE v_running_max INT DEFAULT 0;


  DECLARE cur CURSOR FOR
    SELECT name, marks
    FROM students
    ORDER BY id;   -- or ORDER BY marks, up to you


  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;


  OPEN cur;


  read_loop: LOOP
    FETCH cur INTO v_name, v_marks;
    IF done = 1 THEN
```

```
      LEAVE read_loop;
    END IF;

    IF v_marks > v_running_max THEN
      SET v_running_max = v_marks;
    END IF;

    SELECT v_name AS name,
           v_marks AS marks,
           v_running_max AS running_max_marks;
  END LOOP;

  CLOSE cur;
END $$

DELIMITER ;

CALL running_max_marks();

-- Concept 2 — Stored Procedures (8 Questions)

-- Q1. Create a procedure `show_students()` to display all students.
DELIMITER $$
```

```
CREATE PROCEDURE show_students()
BEGIN
  SELECT * FROM students;
END $$

DELIMITER ;

CALL show_students();
```

```
-- Q2. Write a procedure `add_student(name, marks)` to insert a student.
DELIMITER $$

CREATE PROCEDURE add_student(IN p_name VARCHAR(50), IN p_marks INT)
BEGIN
  INSERT INTO students(name, marks)
  VALUES (p_name, p_marks);
END $$

DELIMITER ;

CALL add_student('Khushi', 78);
```

```
CALL show_students();



-- Q3. Write a procedure `update_marks(id, new_marks)` to update marks.

DELIMITER $$


CREATE PROCEDURE update_marks(IN p_id INT, IN p_new_marks INT)

BEGIN

  UPDATE students

  SET marks = p_new_marks

  WHERE id = p_id;

END $$


DELIMITER ;


CALL update_marks(1, 95);

CALL show_students();



-- Q4. Write a procedure `delete_student(id)` and print rows deleted.

DELIMITER $$


CREATE PROCEDURE delete_student(IN p_id INT)
```

```
BEGIN
  DECLARE v_rows_deleted INT;

  DELETE FROM students
  WHERE id = p_id;

  SET v_rows_deleted = ROW_COUNT();

  SELECT v_rows_deleted AS rows_deleted;
END $$

DELIMITER ;

CALL delete_student(2);
CALL show_students();


-- Q5. Write a procedure `check_even_odd(num)`.
DELIMITER $$

CREATE PROCEDURE check_even_odd(IN p_num INT)
BEGIN
  IF (p_num % 2 = 0) THEN
```

```
    SELECT CONCAT(p_num, ' is EVEN') AS result;
  ELSE
    SELECT CONCAT(p_num, ' is ODD') AS result;
  END IF;
END $$

DELIMITER ;

CALL check_even_odd(10);
CALL check_even_odd(7);
```

```
-- Q6. Write a procedure to calculate Net Salary (HRA 20%, DA 10%, PF 5%).
DELIMITER $$

CREATE PROCEDURE calc_net_salary(
    IN  p_basic   DECIMAL(10,2),
    OUT p_hra     DECIMAL(10,2),
    OUT p_da      DECIMAL(10,2),
    OUT p_pf      DECIMAL(10,2),
    OUT p_netpay  DECIMAL(10,2)
)
```

```
BEGIN
  SET p_hra    = p_basic * 0.20;
  SET p_da     = p_basic * 0.10;
  SET p_pf     = p_basic * 0.05;
  SET p_netpay = p_basic + p_hra + p_da - p_pf;
END $$

DELIMITER ;

CALL calc_net_salary(30000, @hra, @da, @pf, @net);
SELECT @hra AS HRA, @da AS DA, @pf AS PF, @net AS Net_Salary;

-- Q7. Write a transaction-based procedure `transfer_amount(from_acc, to_acc, amount)`.
CREATE TABLE accounts (
  acc_no   INT PRIMARY KEY,
  holder   VARCHAR(50),
  balance  DECIMAL(10,2)
);

INSERT INTO accounts VALUES
(101, 'Amit', 10000),
(102, 'Bhavin', 8000);
```

```
SELECT * FROM accounts;


DELIMITER $$


CREATE PROCEDURE transfer_amount(
    IN p_from_acc INT,
    IN p_to_acc   INT,
    IN p_amount   DECIMAL(10,2)
)
BEGIN
  DECLARE v_from_balance DECIMAL(10,2);


  -- Error handler: if anything fails, rollback
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    SELECT 'Error occurred. Transaction rolled back.' AS message;
  END;


  START TRANSACTION;


  -- Check balance of source account
  SELECT balance INTO v_from_balance
```

```
  FROM accounts
  WHERE acc_no = p_from_acc
  FOR UPDATE;

  IF v_from_balance < p_amount THEN
    ROLLBACK;
    SELECT 'Insufficient balance. Transaction cancelled.' AS message;
  ELSE
    -- Debit from source
    UPDATE accounts
    SET balance = balance - p_amount
    WHERE acc_no = p_from_acc;

    -- Credit to destination
    UPDATE accounts
    SET balance = balance + p_amount
    WHERE acc_no = p_to_acc;

    COMMIT;
    SELECT 'Amount transferred successfully.' AS message;
  END IF;
END $$
```

```
DELIMITER ;


CALL transfer_amount(101, 102, 2000);

SELECT * FROM accounts;



-- Q8. Write a procedure using CONTINUE HANDLER for duplicate entry error.

DELIMITER $$


CREATE PROCEDURE add_student_safe(IN p_name VARCHAR(50), IN p_marks INT)

BEGIN

  DECLARE v_msg VARCHAR(100) DEFAULT 'Inserted successfully.';


  DECLARE CONTINUE HANDLER FOR 1062

  BEGIN

    SET v_msg = 'Duplicate entry. Student not inserted.';

  END;


  INSERT INTO students(name, marks)

  VALUES (p_name, p_marks);


  SELECT v_msg AS message;

END $$
```

```
DELIMITER ;

CALL add_student_safe('Kiran', 88);
CALL add_student_safe('Kiran', 90);

-- Concept 3 — Stored Functions (8 Questions)

-- Q1. Write a function `square(n)`.
DELIMITER $$

CREATE FUNCTION square(n INT)
RETURNS INT
DETERMINISTIC
BEGIN
  RETURN n * n;
END $$

DELIMITER ;

SELECT square(5);
```

```
-- Q2. Write a function `add_two(a, b)`.

DELIMITER $$


CREATE FUNCTION add_two(a INT, b INT)

RETURNS INT

DETERMINISTIC

BEGIN

  RETURN a + b;

END $$


DELIMITER ;


SELECT add_two(11, 22);



-- Q3. Write a function `get_grade(marks)`.

DELIMITER $$


CREATE FUNCTION get_grade(marks INT)

RETURNS CHAR(1)

DETERMINISTIC

BEGIN

  IF marks >= 80 THEN
```

```
    RETURN 'A';
  ELSEIF marks >= 60 THEN
    RETURN 'B';
  ELSE
    RETURN 'C';
  END IF;
END $$

DELIMITER ;

SELECT get_grade(78);

-- Q4. Write a function `calc_tax(salary)` (18% tax).
DELIMITER $$

CREATE FUNCTION calc_tax(salary DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
  RETURN salary * 0.18;
END $$

DELIMITER ;
```

```
SELECT calc_tax(50000);


-- Q5. Write a function `yearly_salary(monthly_salary)`.

DELIMITER $$


CREATE FUNCTION yearly_salary(monthly_salary DECIMAL(10,2))

RETURNS DECIMAL(10,2)

DETERMINISTIC

BEGIN

  RETURN monthly_salary * 12;

END $$


DELIMITER ;


SELECT yearly_salary(35000);





-- Q6. Write a function `get_marks(id)`.

DELIMITER $$


CREATE FUNCTION get_marks(stud_id INT)
```

```
RETURNS INT

DETERMINISTIC

BEGIN

  DECLARE m INT;


  SELECT marks INTO m

  FROM students

  WHERE id = stud_id;


  RETURN m;

END $$


DELIMITER ;


SELECT get_marks(11);


-- Q7. Write a function `is_pass(marks)` returning PASS/FAIL.

DELIMITER $$


CREATE FUNCTION is_pass(marks INT)

RETURNS VARCHAR(10)

DETERMINISTIC

BEGIN
```

```
  IF marks >= 40 THEN
    RETURN 'PASS';
  ELSE
    RETURN 'FAIL';
  END IF;
END $$

DELIMITER ;

SELECT is_pass(72);
SELECT is_pass(22);

-- Q8. Write a function that returns number of students scoring above 80.
DELIMITER $$

CREATE FUNCTION count_above_80()
RETURNS INT
DETERMINISTIC
BEGIN
  DECLARE c INT;

  SELECT COUNT(*) INTO c
  FROM students
```

```
  WHERE marks > 80;

  RETURN c;
END $$

DELIMITER ;

SELECT count_above_80();


-- Concept 4 — Database Triggers (8 Questions)

-- Q1. Write BEFORE INSERT trigger to uppercase name.
DELIMITER $$

CREATE TRIGGER before_student_insert_upper
BEFORE INSERT ON students
FOR EACH ROW
BEGIN
  SET NEW.name = UPPER(NEW.name);
END $$

DELIMITER ;
```

```
-- Test

INSERT INTO students(name, marks) VALUES ('hp', 83);

SELECT * FROM students;




-- Q2. Write AFTER INSERT trigger to log student entry in `student_log`.

CREATE TABLE student_log (

  log_id      INT PRIMARY KEY AUTO_INCREMENT,

  student_id  INT,

  student_name VARCHAR(50),

  action      VARCHAR(20),

  log_time    DATETIME

);


DELIMITER $$


CREATE TRIGGER after_student_insert_log

AFTER INSERT ON students

FOR EACH ROW

BEGIN

  INSERT INTO student_log(student_id, student_name, action, log_time)
```

```
  VALUES (NEW.id, NEW.name, 'INSERT', NOW());
END $$

DELIMITER ;

-- Test
INSERT INTO students(name, marks) VALUES ('Rahul', 75);
SELECT * FROM student_log;


-- Q3. Write BEFORE UPDATE trigger preventing marks > 100.
DELIMITER $$

CREATE TRIGGER before_student_update_limit_marks
BEFORE UPDATE ON students
FOR EACH ROW
BEGIN
  IF NEW.marks > 100 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Marks cannot be greater than 100';
  END IF;
END $$
```

```
DELIMITER ;


-- Test
UPDATE students
SET marks = 120
WHERE id = 11;


-- Q4. Write BEFORE DELETE trigger preventing delete of ‘Admin’.
DELIMITER $$


CREATE TRIGGER before_student_delete_protect_admin
BEFORE DELETE ON students
FOR EACH ROW
BEGIN
  IF OLD.name = 'Admin' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Admin record cannot be deleted';
  END IF;
END $$


DELIMITER ;


-- Test
```

```
DELETE FROM students

WHERE name = 'Admin';


-- insert into students(name, marks) values('admin', 22);



-- Q5. Write AFTER UPDATE trigger logging old and new marks.

CREATE TABLE marks_log (

  log_id      INT PRIMARY KEY AUTO_INCREMENT,

  student_id  INT,

  old_marks   INT,

  new_marks   INT,

  changed_at  DATETIME

);


DELIMITER $$


CREATE TRIGGER after_student_update_log_marks

AFTER UPDATE ON students

FOR EACH ROW

BEGIN

 IF OLD.marks <> NEW.marks THEN

    INSERT INTO marks_log(student_id, old_marks, new_marks, changed_at)
```

```
    VALUES (OLD.id, OLD.marks, NEW.marks, NOW());
  END IF;
END $$

DELIMITER ;

-- Test
UPDATE students
SET marks = 99
WHERE id = 11;

SELECT * FROM marks_log;

-- Q6. Write BEFORE INSERT trigger to auto-calc grade.
DELIMITER $$

CREATE TRIGGER before_student_insert_set_grade
BEFORE INSERT ON students
FOR EACH ROW
BEGIN
  IF NEW.marks >= 80 THEN
    SET NEW.grade = 'A';
  ELSEIF NEW.marks >= 60 THEN
```

```
    SET NEW.grade = 'B';
  ELSE
    SET NEW.grade = 'C';
  END IF;
END $$

DELIMITER ;

-- Test
INSERT INTO students(name, marks) VALUES ('Disha', 65);
SELECT * FROM students;

-- Q7. Write BEFORE UPDATE trigger preventing marks decrease.
DELIMITER $$

CREATE TRIGGER before_student_update_no_decrease
BEFORE UPDATE ON students
FOR EACH ROW
BEGIN
  IF NEW.marks < OLD.marks THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Marks cannot be decreased';
  END IF;
```

```
END $$

DELIMITER ;

-- Test
UPDATE students
SET marks = 40
WHERE id = 11;

-- Q8. Write trigger to reduce product stock after order insert.

CREATE TABLE products (
  product_id   INT PRIMARY KEY,
  product_name VARCHAR(50),
  stock        INT
);

CREATE TABLE orders (
  order_id    INT PRIMARY KEY AUTO_INCREMENT,
  product_id  INT,
  qty         INT
);
```

```
DELIMITER $$

CREATE TRIGGER after_order_insert_reduce_stock
AFTER INSERT ON orders
FOR EACH ROW
BEGIN
  UPDATE products
  SET stock = stock - NEW.qty
  WHERE product_id = NEW.product_id;
END $$

DELIMITER ;

-- Test
INSERT INTO products VALUES (2, 'Pen', 100);

INSERT INTO orders(product_id, qty) VALUES (1, 5);
SELECT * FROM products;
```